```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import Dense, LSTM, Dropout


# Load data
stock_symbol = 'TSLA'
start_date = '2015-01-01'
end_date = '2024-10-10'

data = yf.download(stock_symbol, start=start_date, end=end_date)
data = data[['Close']]
data.tail()
```

```
[*********************100%***********************]  1 of 1 completed
```

Date	Close
2024-10-03	240.660004
2024-10-04	250.080002
2024-10-07	240.830002
2024-10-08	244.500000
2024-10-09	241.050003

```
# Scale the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)


train_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[:train_size]

X_train = []
y_train = []

for i in range(60, len(train_data)):
    X_train.append(train_data[i-60:i, 0])
```

```
        y_train.append(train_data[i, 0])

X_train, y_train = np.array(X_train), np.array(y_train)


X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))


model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=1))  # Output layer

model.compile(optimizer='adam', loss='mean_squared_error')
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a l
  super().__init__(**kwargs)
```

```
model.fit(X_train, y_train, epochs=30, batch_size=32)
```

```
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 0.0025
Epoch 3/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 5s 77ms/step - loss: 0.0025
Epoch 4/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 60ms/step - loss: 0.0018
Epoch 5/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 47ms/step - loss: 0.0016
Epoch 6/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 6s 58ms/step - loss: 0.0016
Epoch 7/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 47ms/step - loss: 0.0018
Epoch 8/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 0.0014
Epoch 9/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 7s 78ms/step - loss: 0.0015
Epoch 10/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 5s 80ms/step - loss: 0.0018
Epoch 11/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 48ms/step - loss: 0.0017
Epoch 12/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 5s 52ms/step - loss: 0.0014
```

```
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 0.0016
Epoch 15/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 5s 47ms/step - loss: 0.0013
Epoch 16/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 71ms/step - loss: 0.0014
Epoch 17/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 48ms/step - loss: 0.0013
Epoch 18/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - loss: 0.0013
Epoch 19/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 59ms/step - loss: 0.0011
Epoch 20/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 71ms/step - loss: 0.0012
Epoch 21/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 0.0013
Epoch 22/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 5s 46ms/step - loss: 0.0012
Epoch 23/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 6s 64ms/step - loss: 9.8295e-04
Epoch 24/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 52ms/step - loss: 8.8800e-04
Epoch 25/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 6s 67ms/step - loss: 0.0010
Epoch 26/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 56ms/step - loss: 0.0013
Epoch 27/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 5s 48ms/step - loss: 0.0012
Epoch 28/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 6s 67ms/step - loss: 0.0010
Epoch 29/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 4s 47ms/step - loss: 0.0011
Epoch 30/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 0.0012
<keras.src.callbacks.history.History at 0x7a08dffcbd60>
```

```
# Prepare test data
test_data = scaled_data[train_size - 60:]
X_test = []
y_test = data['Close'][train_size:].values

for i in range(60, len(test_data)):
    X_test.append(test_data[i-60:i, 0])

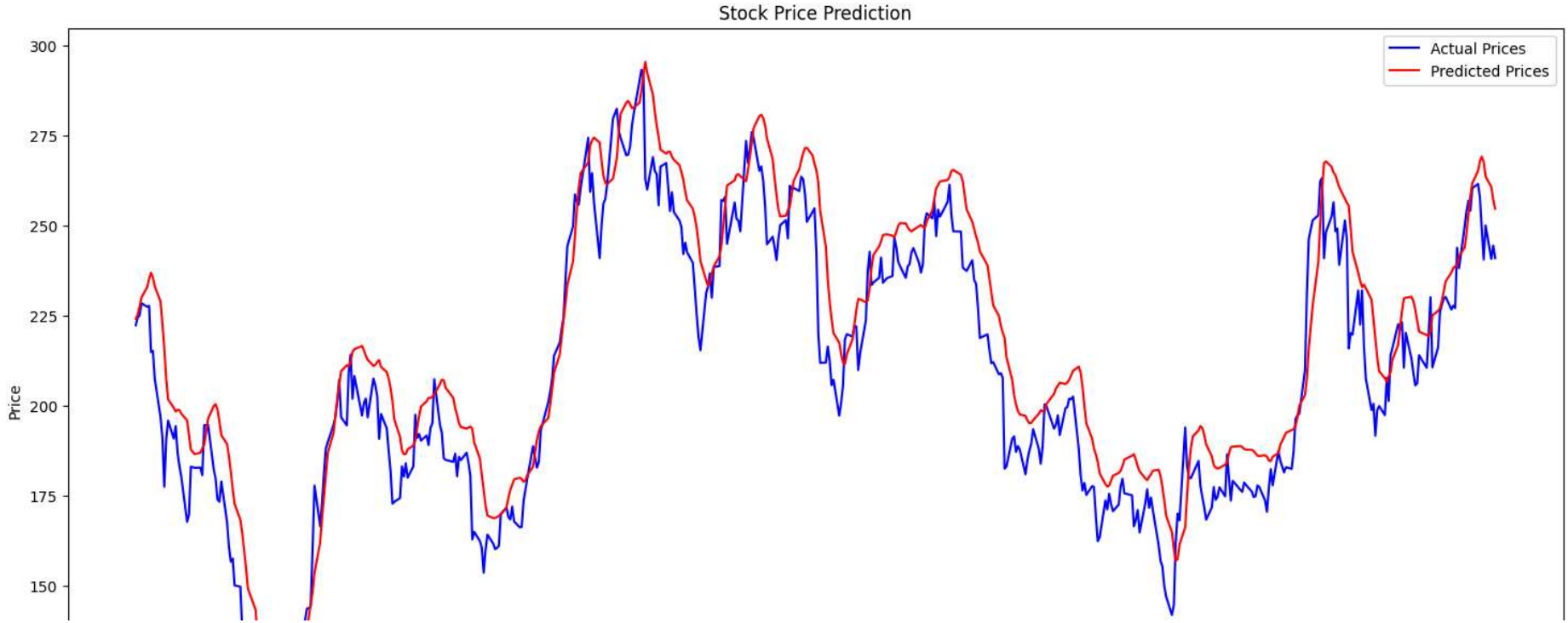
X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))


# Predict
predictions = model.predict(X_test)
```

```
predictions = scaler.inverse_transform(predictions)
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step
```

```
# Plot the results
plt.figure(figsize=(18, 9))
plt.plot(data.index[train_size:], y_test, color='blue', label='Actual Prices')
plt.plot(data.index[train_size:], predictions, color='red', label='Predicted Prices')
plt.title('Stock Price Prediction')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()
```


Stock Price Prediction
Actual Prices
Predicted Prices
Price
300
275
250
225
200
175
150